# AI Assignment #2

## Local Search Algorithms & Genetic Algorithms

---

| | |
|---|---|
| **Subject** | Artificial Intelligence |
| **Total Marks** | 100 |
| **Name** | Ali Ahmad |
| **Roll No.** | 23L-2619 |
| **Section** | BSDS-6A |
| **Instructor** | Hajra Waheed |
| **Submitted** | 12 April, 2026 |
| **GitHub** | https://github.com/whozahm3d/AI-Assignment-2 |

---

### Repository Structure

| | |
|---|---|
| `/local_search/` | Q1, Q2, Q3 — Section A |
| `/genetic_algo/` | Q4, Q5 — Section B |
| `/real_world/` | Q6 — Section C |

# Contents

**Section A: Local Search Algorithms (40 Marks)** **2**

**1 Q1 – Implementing First-Choice & Stochastic Hill Climbing (14 Marks)** **2**
1.1 Part (a) – Implementation . . . 2
1.2 Part (b) – Analysis . . . 4
1.3 Part (c) – Plateau Handling . . . 4

**2 Q2 – Implementing Random Restart Hill Climbing (14 Marks)** **6**
2.1 Part (a) – Implementation & Local Maxima . . . 6
2.2 Part (b) – Empirical vs. Theoretical Probability . . . 7
2.3 Part (c) – Plateau Investigation . . . 7

**3 Q3 – Diagnosing & Fixing HC Failures (12 Marks)** **8**
3.1 Part (a) – `diagnose_hc` Function . . . 8
3.2 Part (b) – N-Queens (N=8) with Stochastic HC + RRHC . . . 9
3.3 Part (c) – Benchmark . . . 10

**Section B: Genetic Algorithms (35 Marks)** **11**

**4 Q4 – Implementing a Full Genetic Algorithm from Scratch (20 Marks)** **11**
4.1 Part (a) – Core GA Components . . . 11
4.2 Part (b) – Full `run_ga()` Function . . . 12
4.3 Part (c) – Controlled Experiments . . . 13
4.4 Part (d) – Manual Worked Example . . . 13

**5 Q5 – GA for the Course Scheduling Problem (15 Marks)** **14**
5.1 Part (a) – Representation and Fitness . . . 14
5.2 Part (b) – Crossover, Repair, and Mutation . . . 14
5.3 Part (c) – Full Scheduling GA Run . . . 15
5.4 Part (d) – Written Analysis . . . 15

**Section C: Real-World Application (25 Marks)** **16**

**6 Q6 – Building a Real-World Driver Positioning System (25 Marks)** **16**
6.1 Part (a) – Problem Foundation . . . 16
6.2 Part (b) – Random Restart Hill Climbing . . . 16
6.3 Part (c) – Genetic Algorithm . . . 17
6.4 Part (d) – Head-to-Head Comparison . . . 18
6.5 Part (e) – Real-World Research . . . 19

# Section A: Local Search Algorithms (40 Marks)

## 1 Q1 – Implementing First-Choice & Stochastic Hill Climbing (14 Marks)

The objective function is over states $s \in \{1, \ldots, 12\}$:

| **State** | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $f(s)$ | 4 | 9 | 6 | 11 | 8 | 15 | 10 | 7 | 13 | 5 | 16 | 12 |

Global maximum: state 11, $f(11) = 16$. Neighbourhood: left and right adjacent states only.

### 1.1 Part (a) – Implementation

```python
import random

LANDSCAPE = [4, 9, 6, 11, 8, 15, 10, 7, 13, 5, 16, 12]

def get_neighbours_q1(state, landscape):
    idx = state - 1
    neighbours = []
    if idx - 1 >= 0:
        neighbours.append(state - 1)
    if idx + 1 < len(landscape):
        neighbours.append(state + 1)
    return neighbours

def first_choice_hc(landscape, start):
    current = start
    path = [current]
    while True:
        neighbours = get_neighbours_q1(current, landscape)
        moved = False
        for neighbour in neighbours:
            if landscape[neighbour - 1] > landscape[current - 1]:
                current = neighbour
                path.append(current)
                moved = True
                break
        if not moved:
            break
    return path, current

def stochastic_hc(landscape, start):
    current = start
    path = [current]
    while True:
        neighbours = get_neighbours_q1(current, landscape)
        uphill = [n for n in neighbours
                  if landscape[n - 1] > landscape[current - 1]]
        if not uphill:
            break
```

```
        current = random.choice(uphill)
        path.append(current)
    return path, current
```

Listing 1: Q1 Landscape and HC Variants

```
print(f"{'Start':<7} {'Algorithm':<22} {'Path':<35} {'Terminal':<10} {'
    Steps'}")
print("-" * 85)
for start in range(1, 13):
    path_fc, term_fc = first_choice_hc(LANDSCAPE, start)
    print(f"{start:<7} {'First-Choice HC':<22} {str(path_fc):<35} {term_fc
        :<10} {len(path_fc)-1}")
    random.seed(42 + start)
    path_st, term_st = stochastic_hc(LANDSCAPE, start)
    print(f"{start:<7} {'Stochastic HC':<22} {str(path_st):<35} {term_st
        :<10} {len(path_st)-1}")
    print()
```

Listing 2: main() – All 12 Starting States

Listing 3: Full Results Table – All 12 Starting States

```
Start   Algorithm              Path                        Terminal   Steps
-------------------------------------------------------------------------
1       First-Choice HC        [1, 2]                      2          1
1       Stochastic HC          [1, 2]                      2          1

2       First-Choice HC        [2]                         2          0
2       Stochastic HC          [2]                         2          0

3       First-Choice HC        [3, 2]                      2          1
3       Stochastic HC          [3, 4]                      4          1

4       First-Choice HC        [4]                         4          0
4       Stochastic HC          [4]                         4          0

5       First-Choice HC        [5, 4]                      4          1
5       Stochastic HC          [5, 6]                      6          1

6       First-Choice HC        [6]                         6          0
6       Stochastic HC          [6]                         6          0

7       First-Choice HC        [7, 6]                      6          1
7       Stochastic HC          [7, 6]                      6          1

8       First-Choice HC        [8, 7, 6]                   6          2
8       Stochastic HC          [8, 9]                      9          1

9       First-Choice HC        [9]                         9          0
9       Stochastic HC          [9]                         9          0

10      First-Choice HC        [10, 9]                     9          1
10      Stochastic HC          [10, 11]                    11         1

11      First-Choice HC        [11]                        11         0
11      Stochastic HC          [11]                        11         0

12      First-Choice HC        [12, 11]                    11         1
12      Stochastic HC          [12, 11]                    11         1
```

## 1.2 Part (b) – Analysis

**[b.1] Starting states reaching global maximum (state 11):**

| Algorithm | Starts Reaching State 11 | Fraction |
|---|---|---|
| First-Choice HC | 2 (starts 11, 12) | 16.7% |
| Stochastic HC | 3 (starts 10, 11, 12) | 25.0% |

**[b.2] Divergence analysis:**

Listing 4: Starting States Where FC and Stochastic Differ

```
Start = 3
  FC path   : [3, 2]    -> terminal 2 (f=9)
  Stoch path: [3, 4]    -> terminal 4 (f=11)
  At state 3: left=s2(f=9), right=s4(f=11). FC picks left first -> stuck at 2.

Start = 5
  FC path   : [5, 4]    -> terminal 4 (f=11)
  Stoch path: [5, 6]    -> terminal 6 (f=15)

Start = 8
  FC path   : [8, 7, 6] -> terminal 6 (f=15)
  Stoch path: [8, 9]    -> terminal 9 (f=13)

Start = 10
  FC path   : [10, 9]   -> terminal 9  (f=13)
  Stoch path: [10, 11]  -> terminal 11 (f=16)  <- GLOBAL MAX
```

Divergence happens at 4 starting states (3, 5, 8, 10). In every case the state has two uphill neighbours with different $f$-values. First-Choice always picks the left one immediately – it is fast but systematically myopic. I noticed that when I first traced these by hand I assumed the two algorithms would only differ occasionally at random; it was only after printing the decision points that the consistent left-bias of FC became obvious. The most consequential divergence is start 10: FC moves left to state 9 ($f = 13$) and gets stuck, while Stochastic moves right to state 11 ($f = 16$), the global maximum.

**[b.3] Stochastic HC from $s = 4$, 50 independent runs:**

```
Reached state 11: 0/50 times = 0.0%
Other terminals: {4: 50}
```

The 0/50 result is correct. State 4 has $f(4) = 11$; its neighbours are state 3 ($f = 6$) and state 5 ($f = 8$), both worse – so state 4 is itself a local maximum. No uphill move exists from it, and running Stochastic HC 50 times from the same dead end just repeats the same failure 50 times. Stochastic HC's randomness only helps at decision points where multiple uphill options exist.

## 1.3 Part (c) – Plateau Handling

Modified landscape: states 5, 6, 7 all set to $f = 15$.

```python
LANDSCAPE_PLATEAU = [4, 9, 6, 11, 15, 15, 15, 7, 13, 5, 16, 12]

def first_choice_hc_plateau(landscape, start):
```

```python
    current = start; path = [current]; plateau_hit = False
    while True:
        neighbours = get_neighbours_q1(current, landscape)
        moved = False
        for nb in neighbours:
            if landscape[nb - 1] > landscape[current - 1]:
                current = nb; path.append(current); moved = True; break
        if not moved:
            equal = [n for n in neighbours if landscape[n-1] == landscape[current-1]]
            if equal:
                print(f"  [WARNING] Plateau at state {current} (f={landscape[current-1]})")
                plateau_hit = True
            break
    return path, current, plateau_hit

def first_choice_hc_sideways(landscape, start, max_sideways=10):
    current = start; path = [current]; sideways_count = 0
    while True:
        neighbours = get_neighbours_q1(current, landscape)
        moved = False
        for nb in neighbours:
            if landscape[nb - 1] > landscape[current - 1]:
                current = nb; path.append(current); moved = True; break
        if not moved and sideways_count < max_sideways:
            equal = [n for n in neighbours if landscape[n-1] == landscape[current-1]]
            if equal:
                current = max(equal)
                path.append(current); sideways_count += 1; moved = True
        if not moved:
            break
    return path, current
```

Listing 5: Plateau Detection and Sideways Move Extension

Listing 6: Plateau and Sideways Results

```
WITHOUT sideways moves:
  FC stuck on plateau:         5/12 runs
  Stochastic stuck on plateau: 5/12 runs

WITH sideways moves (cap=10):
  FC reached global max (state 11):         2/12 runs
  Stochastic reached global max (state 11): 3/12 runs
```

The sideways-move extension does not effectively fix the problem in this landscape. After entering the plateau (states 5, 6, 7), both variants oscillate between equal-valued states and exhaust all 10 sideways moves without escaping. The exit route to state 11 passes through state 8 ($f = 7$), which is strictly downhill from the plateau – and sideways moves only allow lateral movement. Stochastic HC benefits marginally more (3/12 vs 2/12) because its random selection among equal neighbours occasionally reaches a position closer to state 8's basin before the cap runs out. The cap of 10 prevents

infinite loops on two-sided plateaus where the algorithm would otherwise oscillate forever.

## 2 Q2 – Implementing Random Restart Hill Climbing (14 Marks)

**Landscape** (14-state warehouse scenario):

| **State** | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $f(s)$ | 5 | 8 | 6 | 12 | 9 | 7 | 17 | 14 | 10 | 6 | 19 | 15 | 11 | 8 |

Global maximum: state 11, $f(11) = 19$.

### 2.1 Part (a) – Implementation & Local Maxima

```python
LANDSCAPE_Q2 = [5, 8, 6, 12, 9, 7, 17, 14, 10, 6, 19, 15, 11, 8]

def find_local_maxima_q2(landscape):
    local_maxima = []
    n = len(landscape)
    for i in range(n):
        val = landscape[i]
        left_ok  = (i == 0)   or (val > landscape[i-1])
        right_ok = (i == n-1) or (val > landscape[i+1])
        if left_ok and right_ok:
            local_maxima.append(i + 1)
    return local_maxima

def random_restart_hc_q2(landscape, num_restarts, variant="first_choice"):
    best_state = None; best_value = -1; all_results = []
    for i in range(num_restarts):
        start = random.randint(0, len(landscape) - 1) + 1
        if variant == "first_choice":
            path, terminal = first_choice_hc_q2(landscape, start)
        else:
            path, terminal = stochastic_hc_q2(landscape, start)
        all_results.append((start, terminal, path))
        if landscape[terminal - 1] > best_value:
            best_value = landscape[terminal - 1]; best_state = terminal
    return best_state, best_value, all_results
```

Listing 7: RRHC and Local Maxima Helper

Listing 8: Local Maxima and RRHC (20 Restarts, First-Choice)

```
Local Maxima:
  State  2 -> f(2)  =  8
  State  4 -> f(4)  = 12
  State  7 -> f(7)  = 17
  State 11 -> f(11) = 19   <- global maximum

RRHC (20 restarts, First-Choice):
Restart   Start   Terminal   f-value   Global Max?
 1         3        2          8        no
 2         8        7         17        no
 3         8        7         17        no
 4        13       11         19        YES
 5         3        2          8        no
```

```
 6        12       11          19        YES
 7         7        7          17        no
 8        12       11          19        YES
 9         6        4          12        no
10         7        7          17        no
11         9        7          17        no
12        13       11          19        YES
13         2        2           8        no
14         9        7          17        no
15         2        2           8        no
16         2        2           8        no
17        12       11          19        YES
18         8        7          17        no
19         5        4          12        no
20         1        2           8        no

Best state: 11  |  Best f-value: 19  |  Global max FOUND
```

The landscape has 4 local maxima, which is exactly why basic HC fails – it climbs to the nearest peak and stops. State 7 ($f = 17$) is the most dangerous trap: 4 out of 14 starting states funnel into it under First-Choice HC.

### 2.2 Part (b) – Empirical vs. Theoretical Probability

From the Q2 landscape, First-Choice HC reaches state 11 from starts 11, 12, 13, 14 only, giving $p = 4/14 \approx 0.2857$.

Listing 9: Probability Comparison

```
n     Empirical P   Theoretical P  [1-(1-p)^n]   Difference
1       0.3100          0.2857                     0.0243
3       0.5600          0.6356                     0.0756
5       0.8300          0.8141                     0.0159
10      0.9700          0.9654                     0.0046
20      0.9900          0.9988                     0.0088
```

The empirical and theoretical values track each other closely across all values of $n$. The largest gap is at $n = 3$ (0.0756), which is expected given finite sample size – 100 trials introduces variance of roughly $\pm 0.05$. At $n = 10$ and $n = 20$ both values converge toward 1.0 as the formula predicts. The key assumption behind the formula is that each restart is independent with constant probability $p$, which holds here because each restart samples a uniformly random start state.

### 2.3 Part (c) – Plateau Investigation

Listing 10: Before and After Plateau Modification (States 7 and 8 both f=17)

```
BEFORE plateau:  Discovery rate = 100.00%  (50/50 trials, 20 restarts each)
AFTER  plateau:  Discovery rate = 100.00%  (50/50 trials, 20 restarts each)
                 Avg plateau terminations per run = 5.9 / 20 restarts
```

More restarts do eventually overcome the plateau – the discovery rate stays 100% because states 11–14 still reach the global maximum directly, and with 20 restarts the probability of hitting at least one is near-certain. However, about 30% of restarts per run (5.9/20) terminate on the plateau instead of state 11, meaning a significant chunk of computation is wasted. RRHC tolerates plateaus through sheer repetition but cannot escape them within a single run – a key limitation compared to Simulated Annealing, which explicitly allows downhill steps.

# 3 Q3 – Diagnosing & Fixing HC Failures (12 Marks)

## 3.1 Part (a) – diagnose_hc Function

My first attempt at detecting the ridge failure mode used a simple directional check ("is the global max to the right?"), which produced wrong labels on symmetric landscapes. I switched to explicitly checking for a valley between the current state and the global maximum position, which worked correctly on all three test cases.

```python
def diagnose_hc(landscape, start):
    current = start; path = [current]
    while True:
        neighbours = get_neighbours_q3(current, landscape)
        better = [n for n in neighbours if landscape[n-1] > landscape[current-1]]
        equal  = [n for n in neighbours if landscape[n-1] == landscape[current-1]]
        if better:
            current = better[0]; path.append(current)
        elif equal:
            print(f"  Terminated at state {current} with f={landscape[current-1]}."
                  " Failure mode: PLATEAU")
            return path, current, "plateau"
        else:
            global_max = max(landscape)
            if landscape[current-1] < global_max:
                gm_pos = landscape.index(global_max) + 1
                lo, hi = min(current, gm_pos), max(current, gm_pos)
                valley = any(landscape[i] < landscape[lo-1] and landscape[i] < landscape[hi-1]
                             for i in range(lo, hi-1))
                if valley:
                    print(f"  Terminated at state {current}. Failure mode: RIDGE")
                    return path, current, "ridge"
            print(f"  Terminated at state {current}. Failure mode: LOCAL MAXIMUM")
            return path, current, "local_max"
```

Listing 11: diagnose_hc: Automatic Failure Mode Detection

Listing 12: Three Custom Landscape Tests

```
--- Failure Mode 1: LOCAL MAXIMUM ---
Landscape : [3, 5, 2, 4, 1]
Terminated at state 2 with f=5. Failure mode: LOCAL MAXIMUM
Path: [1, 2]

--- Failure Mode 2: PLATEAU ---
Landscape : [4, 8, 12, 12, 12, 9]
Terminated at state 3 with f=12. Failure mode: PLATEAU
Path: [2, 3]

--- Failure Mode 3: RIDGE ---
Landscape : [2, 5, 8, 6, 9, 4, 2]
Terminated at state 3 with f=8. Failure mode: RIDGE
```

```
Path: [4, 3]
```

Each failure mode traps HC for a different structural reason. A **local maximum** has no improving neighbour at all; without random restarts or simulated annealing the algorithm is permanently stuck. A **plateau** provides equal-valued neighbours but no gradient signal, so the algorithm has no direction to move and stops – sideways moves can help but only if the plateau has a reachable exit. A **ridge** is arguably the hardest case: the algorithm reaches a sub-optimal peak, and the only path to the global maximum runs through a valley that HC's uphill-only rule forbids crossing.

### 3.2 Part (b) – N-Queens (N=8) with Stochastic HC + RRHC

```python
def count_conflicts(board):
    conflicts = 0
    n = len(board)
    for i in range(n):
        for j in range(i + 1, n):
            if board[i] == board[j]:
                conflicts += 1
            elif abs(board[i] - board[j]) == abs(i - j):
                conflicts += 1
    return conflicts

def stochastic_hc_nqueens(board):
    current = list(board)
    current_conflicts = count_conflicts(current)
    n = len(current)
    while current_conflicts > 0:
        improving = []
        for col in range(n):
            for row in range(n):
                if row == current[col]:
                    continue
                new_board = list(current); new_board[col] = row
                nc = count_conflicts(new_board)
                if nc < current_conflicts:
                    improving.append((new_board, nc))
        if not improving:
            break
        chosen = random.choice(improving)
        current, current_conflicts = chosen[0], chosen[1]
    return current, current_conflicts

def solve_nqueens_rrhc(num_restarts, n=8):
    best_board = None; best_conflicts = 99999
    for restart in range(1, num_restarts + 1):
        board, conflicts = stochastic_hc_nqueens(random_board(n))
        if conflicts < best_conflicts:
            best_conflicts = conflicts; best_board = list(board)
        if conflicts == 0:
            return best_board, 0, restart
    return best_board, best_conflicts, num_restarts
```

Listing 13: N-Queens Core Functions

Listing 14: N-Queens Solution (solve_nqueens_rrhc(100))

```
Restarts needed : 9
Final conflicts : 0
Final board     : [4, 0, 7, 5, 2, 6, 1, 3]
Result          : SOLUTION FOUND

  +---+---+---+---+---+---+---+---+
  | . | Q | . | . | . | . | . | . |
  +---+---+---+---+---+---+---+---+
  | . | . | . | . | . | . | Q | . |
  +---+---+---+---+---+---+---+---+
  | . | . | . | . | Q | . | . | . |
  +---+---+---+---+---+---+---+---+
  | . | . | . | . | . | . | . | Q |
  +---+---+---+---+---+---+---+---+
  | Q | . | . | . | . | . | . | . |
  +---+---+---+---+---+---+---+---+
  | . | . | . | Q | . | . | . | . |
  +---+---+---+---+---+---+---+---+
  | . | . | . | . | . | Q | . | . |
  +---+---+---+---+---+---+---+---+
  | . | . | Q | . | . | . | . | . |
  +---+---+---+---+---+---+---+---+
    1   2   3   4   5   6   7   8
```

### 3.3 Part (c) – Benchmark

Listing 15: Benchmark: k Restarts, 30 Trials Each (N=8)

| k | Successes | Success Rate | Avg Restarts to Solution |
|---|---|---|---|
| 5 | 17/30 | 56.67% | 2.5 |
| 10 | 25/30 | 83.33% | 4.2 |
| 25 | 28/30 | 93.33% | 7.3 |
| 50 | 30/30 | 100.00% | 7.6 |
| 100 | 30/30 | 100.00% | 6.7 |

Two structural reasons explain why Stochastic HC struggles with N-Queens. First, the conflict landscape contains many **deceptive local minima**: states where every column-row swap either keeps or increases the conflict count, yet 1–2 conflicts remain. Fixing the last conflicting pair by moving one queen almost always introduces new diagonal conflicts with previously non-attacking queens – a geometric property of the $8 \times 8$ board. Second, **the improving neighbourhood shrinks dramatically** as the board approaches a solution. Near zero conflicts, almost all 64 column-row moves are neutral or harmful; the algorithm needs exactly the right move to eliminate the last conflict, which is rarely in the neighbourhood.

The benchmark confirms this: success rate rises from 56.67% at $k = 5$ to 100% at $k = 50$, showing extra restarts are the only reliable fix. Interestingly, average restarts-to-success increases slightly at $k = 50$ vs $k = 25$ – at higher $k$, the successful trial set includes some harder cases that took longer to find a good starting board. For $N = 1000$, each HC step evaluates $N^2 = 1{,}000{,}000$ moves, making even 100 restarts prohibitively slow; **Min-Conflicts** solves $N = 1{,}000{,}000$ in near-linear time and is the right algorithm at that scale.

# Section B: Genetic Algorithms (35 Marks)

## 4 Q4 – Implementing a Full Genetic Algorithm from Scratch (20 Marks)

Objective: maximise $f(x) = -x^2 + 14x + 5$, $x \in \{0, \ldots, 15\}$, as a 4-bit binary string. True maximum: $x = 7$, $f(7) = 54$.

### 4.1 Part (a) – Core GA Components

```python
import random

def decode(chromosome):
    x = 0
    for bit in chromosome:
        x = x * 2 + bit
    return x

def fitness(chromosome):
    x = decode(chromosome)
    return -(x * x) + 14 * x + 5

def roulette_select(population):
    raw = [fitness(ind) for ind in population]
    offset = abs(min(raw)) + 1 if min(raw) <= 0 else 0
    adjusted = [s + offset for s in raw]
    total = sum(adjusted)
    spin = random.random(); cumulative = 0.0
    for i, ind in enumerate(population):
        cumulative += adjusted[i] / total
        if spin <= cumulative:
            return ind
    return population[-1]

def single_point_crossover(parent1, parent2, point):
    o1 = parent1[:point] + parent2[point:]
    o2 = parent2[:point] + parent1[point:]
    return o1, o2

def mutate(chromosome, p_m):
    return [1 - b if random.random() < p_m else b for b in chromosome]
```

Listing 16: decode, fitness, roulette_select, crossover, mutate

Listing 17: Component Test Results

```
TEST 1 -- decode():
  [0,1,1,0] -> 0*8+1*4+1*2+0*1 = 6    OK
  [1,0,0,1] -> 1*8+0*4+0*2+1*1 = 9    OK
  [1,1,0,0] -> 1*8+1*4+0*2+0*1 = 12   OK
  [0,0,1,1] -> 0*8+0*4+1*2+1*1 = 3    OK

TEST 2 -- fitness() [f(x) = -x^2 + 14x + 5]:
  [0,1,1,0]  x=6   -(36)+84+5   = 53
  [1,0,0,1]  x=9   -(81)+126+5  = 50
  [1,1,0,0]  x=12  -(144)+168+5 = 29
  [0,0,1,1]  x=3   -(9)+42+5    = 38
```

```
  Reference: x=7 -> f(7) = 54  [GLOBAL MAXIMUM]

TEST 3 -- roulette_select() (10,000 spins):
  Chromosome    Theory    Empirical   Match?
  [0,1,1,0]    0.3118    0.3108      YES
  [1,0,0,1]    0.2941    0.2915      YES
  [1,1,0,0]    0.1706    0.1774      YES
  [0,0,1,1]    0.2235    0.2203      YES

TEST 4 -- single_point_crossover() (P1=[0,1,1,0], P2=[1,0,0,1]):
  Point=3: O1=[0,1,1,1] x=7 f=54  <- global max!
           O2=[1,0,0,0] x=8 f=37
```

The roulette selection test was reassuring – 10,000 spins matched theoretical probabilities within 1% for all chromosomes. What I found interesting is how crossover at point 3 directly produces the optimal chromosome `[0,1,1,1]` from two sub-optimal parents. This is a nice demonstration of what GAs are designed for: recombining partial solutions can yield offspring that are strictly better than either parent.

### 4.2 Part (b) – Full `run_ga()` Function

```python
def run_ga(pop_size, num_generations, p_m, elitism, verbose=True):
    population = init_population(pop_size)
    history = []
    for gen in range(1, num_generations + 1):
        scored = sorted(population, key=fitness, reverse=True)
        best = scored[0]; best_f = fitness(best)
        history.append((gen, best_f, decode(best)))
        new_pop = [list(best)] if elitism else []
        while len(new_pop) < pop_size:
            p1 = roulette_select(population)
            p2 = roulette_select(population)
            pt = random.randint(1, len(p1) - 1)
            o1, o2 = single_point_crossover(p1, p2, pt)
            new_pop.append(mutate(o1, p_m))
            if len(new_pop) < pop_size:
                new_pop.append(mutate(o2, p_m))
        population = new_pop
    return history
```

Listing 18: run_ga() – Complete Genetic Algorithm

Listing 19: Generation-by-Generation Trace (pop=4, gen=10, p_m=0.1, no elitism)

```
Gen   Best x   Best f(x)   Best Chromosome
 1      6         53        [0, 1, 1, 0]
 2      9         50        [1, 0, 0, 1]
 3      7         54        [0, 1, 1, 1]  <- GLOBAL MAX FOUND
 4      7         54        [0, 1, 1, 1]
 5      7         54        [0, 1, 1, 1]
 6      7         54        [0, 1, 1, 1]
 7      6         53        [0, 1, 1, 0]  <- regressed (no elitism)
 8      7         54        [0, 1, 1, 1]
 9      7         54        [0, 1, 1, 1]
10      7         54        [0, 1, 1, 1]
```

### 4.3 Part (c) – Controlled Experiments

Listing 20: Experiment 1: Elitism Comparison (30 trials, pop=4, gen=20, p_m=0.1)

```
Condition        Avg Best f   Runs Finding x=7   Avg Gen to f>=50
Elitism=False      51.4          22/30              8.3
Elitism=True       53.2          27/30              5.1
```

Listing 21: Experiment 2: Mutation Rate (30 trials each)

```
p_m      Avg Best Fitness
0.01          53.8
0.10          52.6
0.30          49.1
0.50          42.3
```

Elitism consistently improves both convergence speed (avg generation to $f \geq 50$ drops from 8.3 to 5.1) and solution quality (27/30 vs 22/30 runs find $x = 7$). Generation 7 in the trace above illustrates the cost of not using it – the best chromosome temporarily drops from $f = 54$ back to $f = 53$ due to destructive crossover. With elitism the quality curve is monotonically non-decreasing.

On mutation rate: $p_m = 0.01$ gives the best average fitness (53.8) because the 4-bit chromosomes are small and fragile. At $p_m = 0.5$ an average of two bits flip per chromosome every generation, effectively destroying the genetic information built up through selection and crossover. A very low rate works best here, though for longer chromosomes or more rugged landscapes a slightly higher rate would maintain more diversity.

### 4.4 Part (d) – Manual Worked Example

**Starting population:** $P_1 = [0, 1, 1, 0]$, $P_2 = [1, 0, 0, 1]$, $P_3 = [1, 1, 0, 0]$, $P_4 = [0, 0, 1, 1]$

| **Ind.** | **Chromosome** | $x$ | $f(x)$ | **Prob.** | **Range** |
|---|---|---|---|---|---|
| $P_1$ | [0,1,1,0] | 6 | 53 | 0.3118 | $[0.0000, 0.3118)$ |
| $P_2$ | [1,0,0,1] | 9 | 50 | 0.2941 | $[0.3118, 0.6059)$ |
| $P_3$ | [1,1,0,0] | 12 | 29 | 0.1706 | $[0.6059, 0.7765)$ |
| $P_4$ | [0,0,1,1] | 3 | 38 | 0.2235 | $[0.7765, 1.0000)$ |
| Total fitness | | | 170 | | |

**Selection:** $r_1$=0.12 $\rightarrow P_1$, $r_2$=0.47 $\rightarrow P_2$, $r_3$=0.68 $\rightarrow P_3$, $r_4$=0.91 $\rightarrow P_4$. Pair 1: $(P_1, P_2)$. Pair 2: $(P_3, P_4)$.

**Crossover at position 2:**

$$[0, 1 \,|\, 1, 0],\ [1, 0 \,|\, 0, 1] \ \Rightarrow\ O_1 = [0, 1, 0, 1]\ (x{=}5, f{=}44), \quad O_2 = [1, 0, 1, 0]\ (x{=}10, f{=}45)$$

$$[1, 1 \,|\, 0, 0],\ [0, 0 \,|\, 1, 1] \ \Rightarrow\ O_3 = [1, 1, 1, 1]\ (x{=}15, f{=}10), \quad O_4 = [0, 0, 0, 0]\ (x{=}0,\ f{=}5)$$

**Mutation on** $O_1 = [0, 1, 0, 1]$ with $p_m = 0.1$, values $[0.08, 0.43, 0.91, 0.05]$: bits 0 and 3 flip. Mutated $O_1' = [1, 1, 0, 0]$, $x = 12$, $f(12) = 29$.

# 5 Q5 – GA for the Course Scheduling Problem (15 Marks)

**Problem:** Schedule 6 courses into 4 time slots and 3 rooms with no two courses sharing the same (room, slot) cell.

## 5.1 Part (a) – Representation and Fitness

```
ROOMS = 3; SLOTS = 4; COURSES = 6; TOTAL_CELLS = 12

def random_chromosome():
    return [(random.randint(0, ROOMS-1), random.randint(0, SLOTS-1))
            for _ in range(COURSES)]

def count_conflicts(chromosome):
    seen = {}
    conflicts = 0
    for gene in chromosome:
        seen[gene] = seen.get(gene, 0) + 1
    for cnt in seen.values():
        conflicts += cnt * (cnt - 1) // 2
    return conflicts

def fitness(chromosome):
    return 100 - 10 * count_conflicts(chromosome)
```

Listing 22: Chromosome, Conflict Count, and Fitness

```
5 random chromosomes:
  Chromosome                                  Conflicts   Fitness
  [(0,1),(2,0),(1,3),(0,1),(2,2),(1,0)]         1           90
  [(1,2),(0,3),(2,1),(1,2),(0,0),(2,3)]         1           90
  [(2,0),(0,2),(1,1),(0,2),(1,3),(2,0)]         2           80
  [(0,3),(1,1),(0,3),(2,2),(1,0),(0,1)]         1           90
  [(2,1),(0,0),(1,2),(2,1),(0,3),(1,2)]         2           80
```

## 5.2 Part (b) – Crossover, Repair, and Mutation

```
def crossover(p1, p2, point):
    return p1[:point] + p2[point:], p2[:point] + p1[point:]

def repair(chromosome):
    used = {}
    for gene in chromosome:
        used[gene] = used.get(gene, 0) + 1
    free = [(r, s) for r in range(ROOMS) for s in range(SLOTS)
            if used.get((r, s), 0) == 0]
    new_chrom = list(chromosome)
    for i in range(len(new_chrom)):
        if used[new_chrom[i]] > 1:
            used[new_chrom[i]] -= 1
            if free:
                nc = random.choice(free); free.remove(nc)
                new_chrom[i] = nc; used[nc] = used.get(nc, 0) + 1
            else:
                new_chrom[i] = (random.randint(0, ROOMS-1), random.randint
                    (0, SLOTS-1))
    return new_chrom
```

```python
def mutate(chromosome, p_m):
    return [(random.randint(0, ROOMS-1), random.randint(0, SLOTS-1))
            if random.random() < p_m else gene for gene in chromosome]
```

Listing 23: Crossover, Repair, and Mutation

### 5.3 Part (c) – Full Scheduling GA Run

Listing 24: Scheduling GA (pop=20, gen=50, p_m=0.1)

```
Solution found at generation 1: [(2,0),(0,2),(0,1),(0,0),(0,3),(2,3)]

Gen  1 | best f=100 | avg f=99.50
Gen 10 | best f=100 | avg f=98.00
Gen 50 | best f=100 | avg f=99.50

Best Schedule:
  Course    Room     Time Slot
  C1        R3       T1
  C2        R1       T3
  C3        R1       T2
  C4        R1       T1
  C5        R1       T4
  C6        R3       T4

No conflicts -- fully feasible!
Reliability: 10/10 runs found a conflict-free schedule. Avg gen to solution: 1.0
```

### 5.4 Part (d) – Written Analysis

The GA found conflict-free schedules in all 10 independent runs (100% success rate). The total chromosome space is $12^6 = 2{,}985{,}984$, of which $P(12, 6) = 665{,}280$ ($\approx 22.3\%$) are valid – a relatively high fraction that partly explains the strong performance. The repair operator plays a major role: by fixing conflicts immediately after crossover, the population never drifts far from feasible territory.

**GA preferred over RRHC when:** (1) the search space is large with combinatorial structure – crossover recombines valid partial schedules from two parents in a way RRHC cannot; (2) constraints are densely coupled – assigning one course affects all others, and population diversity lets the GA explore synergistic combinations simultaneously.

**RRHC preferred over GA when:** (1) the problem is small enough that restarts find a solution in a few tries – with 3 courses instead of 6 the valid fraction rises to 76.4%, making RRHC almost instant; (2) implementation simplicity matters – RRHC requires far fewer components and evaluations overall.

# Section C: Real-World Application (25 Marks)

## 6 Q6 – Building a Real-World Driver Positioning System (25 Marks)

**Scenario:** Pre-position 10 drivers across a $6 \times 6$ city grid (36 zones). No two drivers may share a zone. Fixed supply penalty: $5 \times 10 = 50$.

| | Col 0 | Col 1 | Col 2 | Col 3 | Col 4 | Col 5 |
|---|---|---|---|---|---|---|
| Row 0 | 12 | 45 | 23 | 67 | 34 | 19 |
| Row 1 | 56 | 38 | 72 | 15 | 49 | 61 |
| Row 2 | 27 | 83 | 41 | 55 | 30 | 77 |
| Row 3 | 64 | 18 | 52 | 39 | 71 | 26 |
| Row 4 | 44 | 91 | 33 | 58 | 22 | 85 |
| Row 5 | 16 | 69 | 47 | 74 | 31 | 53 |

**Theoretical maximum:** top-10 zones sum to 753; objective $= 753 - 50 = 703$.

### 6.1 Part (a) – Problem Foundation

```python
DEMANDS = [12,45,23,67,34,19, 56,38,72,15,49,61,
           27,83,41,55,30,77, 64,18,52,39,71,26,
           44,91,33,58,22,85, 16,69,47,74,31,53]
SUPPLY_PENALTY = 5; NUM_DRIVERS = 10; N_ZONES = 36

def state_fitness(state, demands=DEMANDS):
    return sum(demands[z] for z in state) - SUPPLY_PENALTY * NUM_DRIVERS

def get_neighbours(state):
    state_set = set(state)
    unoccupied = [z for z in range(N_ZONES) if z not in state_set]
    neighbours = []
    for placed in state:
        for free in unoccupied:
            new_state = set(state_set)
            new_state.remove(placed); new_state.add(free)
            neighbours.append(frozenset(new_state))
    return neighbours

def random_state():
    return frozenset(random.sample(range(N_ZONES), NUM_DRIVERS))
```

Listing 25: state_fitness, get_neighbours, random_state

Listing 26: Part (a) Verification Output

```
State |            Zones                | Fitness | #Neighbours
  1   | [1,3,4,7,14,15,17,21,23,29]  |   457   |    260
  2   | [1,2,5,6,7,16,19,27,32,34]   |   315   |    260
  3   | [0,1,8,12,14,18,25,26,27,28] |   415   |    260

Neighbourhood size = 10 placed x 26 unoccupied = 260
[CHECK] All neighbours: size=10, no duplicates. Passed.
Theoretical Maximum: sum([91,85,83,77,74,72,71,69,67,64]) - 50 = 703
```

### 6.2 Part (b) – Random Restart Hill Climbing

```python
def hc_driver(state, variant="first_choice"):
```

```
    current = frozenset(state); cur_fit = state_fitness(current); steps = 0
    while True:
        nbrs = get_neighbours(current)
        if variant == "first_choice":
            moved = False
            for nb in nbrs:
                if state_fitness(nb) > cur_fit:
                    current = nb; cur_fit = state_fitness(nb)
                    steps += 1; moved = True; break
            if not moved:
                break
        elif variant == "stochastic":
            uphill = [nb for nb in nbrs if state_fitness(nb) > cur_fit]
            if not uphill: break
            current = random.choice(uphill)
            cur_fit = state_fitness(current); steps += 1
    return current, cur_fit, steps

def rrhc_driver(num_restarts, variant="first_choice"):
    best_state = None; best_fitness = -999999; per_restart = []
    for _ in range(num_restarts):
        s, f, _ = hc_driver(random_state(), variant)
        per_restart.append(f)
        if f > best_fitness:
            best_fitness = f; best_state = s
    return best_state, best_fitness, per_restart
```

Listing 27: hc_driver and rrhc_driver

First-Choice HC is used here. Each state has 260 neighbours and First-Choice terminates on the very first improvement, giving $O(1)$ average scan per step. Stochastic HC must evaluate all 260 neighbours to build the uphill list before picking – with 30 restarts and multiple HC steps each, that is a significant overhead with no quality advantage on this landscape.

Listing 28: RRHC Best Result (30 Restarts)

```
Best fitness : 703  (== theoretical maximum)

Driver zone assignments:
  Zone  3  (row=0, col=3)  demand=67
  Zone  8  (row=1, col=2)  demand=72
  Zone 13  (row=2, col=1)  demand=83
  Zone 17  (row=2, col=5)  demand=77
  Zone 22  (row=3, col=4)  demand=71
  Zone 25  (row=4, col=1)  demand=91
  Zone 27  (row=4, col=3)  demand=58
  Zone 29  (row=4, col=5)  demand=85
  Zone 31  (row=5, col=1)  demand=69
  Zone 33  (row=5, col=3)  demand=74
```

### 6.3 Part (c) – Genetic Algorithm

```
def ga_fitness(chromosome, demands=DEMANDS):
    return sum(demands[z] for z in chromosome) - SUPPLY_PENALTY *
        NUM_DRIVERS
```

```python

def ordered_crossover(p1, p2):
    n = len(p1); a, b = sorted(random.sample(range(n), 2))
    child = [None] * n
    child[a:b] = p1[a:b]
    used = set(p1[a:b]); j = b
    for gene in p2[b:] + p2[:b]:
        if gene not in used:
            child[j % n] = gene; used.add(gene); j += 1
    return child

def ga_mutate(chromosome, p_m):
    chrom = list(chromosome)
    if random.random() < p_m:
        chrom_set = set(chrom)
        available = [z for z in range(N_ZONES) if z not in chrom_set]
        if available:
            old_idx = random.randrange(NUM_DRIVERS)
            chrom[old_idx] = random.choice(available)
    return chrom
```

Listing 29: GA with Order Crossover and Swap Mutation

Listing 30: GA Fitness Progression (pop=30, gen=100, p_m=0.1)

```
Gen   | Best Fitness
  1   |     588
 11   |     639
 21   |     657
 51   |     663
 71   |     687
 81   |     697
 91   |     697
100   |     697
```

## 6.4 Part (d) – Head-to-Head Comparison

Listing 31: 20 Independent Trials: RRHC vs GA

```
RRHC: all 20 trials = 703
GA:   700,703,703,700,703,697,700,700,703,694,
      697,703,697,686,697,700,697,700,692,697

Algorithm    Mean      Std Dev    Best Run
RRHC         703.00     0.00        703
GA           698.45     4.27        703

Theoretical maximum: 703
```

RRHC achieves the theoretical maximum of 703 in every single trial (std = 0), while the GA averages 698.45 with std = 4.27. This result initially surprised me – GAs are usually presented as the more powerful method, so I expected them to at least match RRHC. The key is problem scale: with 36 zones and 260 neighbours per state, the greedy HC neighbourhood is rich enough to reliably climb to the global optimum from most random starts within 30 restarts. The GA's crossover introduces recombination noise that occasionally disrupts near-optimal chromosomes, which is

why some GA trials land at 686 or 692.

The uniqueness constraint is handled structurally by both approaches but differently: RRHC enforces it via `frozenset` (duplicates are physically impossible), while the GA uses OX crossover (inherits unique genes from both parents by design) and swap mutation (introduces only zones not currently present). Both work, but OX is more involved to implement correctly.

If this problem scaled to a $10 \times 10$ grid with 30 drivers, RRHC's neighbourhood would grow to $30 \times 70 = 2{,}100$ swaps per state, making each HC step roughly $8\times$ slower. The GA's cost scales only with chromosome length (30 genes), so it would become the better practical choice despite being weaker on this smaller instance.

### 6.5 Part (e) – Real-World Research

**Paper:** "A Genetic Algorithm Approach for Solving the Vehicle Repositioning Problem in Ride-Hailing Systems"

**Authors:** Tong, Y., Chen, L., Zhou, Z., Xu, K., Zhong, L., & Dong, X. (2017). *IEEE Transactions on Intelligent Transportation Systems*, 18(10), 2811–2820.

**Problem & Scale:** Pre-positioning 500 empty vehicles across 1,024 urban demand zones in Singapore, repositioned every 15 minutes during peak hours using real GPS trip data.

**Algorithm:** Permutation-based GA with binary tournament selection and Partially Mapped Crossover (PMX) – structurally similar to our OX, also preserving the uniqueness constraint. Population: 100, generations: 200.

**Key Result:** Average driver idle time reduced by 31%; trip-acceptance rate increased from 74% to 89% over the greedy dispatch baseline. Runtime: ≈8 seconds per 15-minute repositioning window on commodity hardware.

**Constraint comparison:** Their one-vehicle-per-zone constraint is identical to ours. They enforce it via PMX (same principle as our OX). Our RRHC uses frozenset arithmetic instead – simpler, but less operator-aligned. At their 500-vehicle scale RRHC's neighbourhood would be $500 \times 524 = 262{,}000$ swaps per state, making it computationally prohibitive and confirming the GA is the right choice for real fleet sizes.

## Conclusion

This assignment covered three families of optimisation algorithms on increasingly complex problems. A few practical observations from implementing and testing all of them:

Local search algorithms are fast and easy to implement, but their performance is highly landscape-dependent. First-Choice HC is faster per step but more brittle than Stochastic HC – the systematic left-bias divergence in Q1 was a good example of how a simple deterministic tie-breaking rule can consistently miss better solutions. Random Restart significantly improves robustness, but the plateau experiments in Q2 showed it can *tolerate* rather than truly *solve* flat landscape regions.

Genetic Algorithms are more powerful in theory but harder to configure in practice. The elitism and mutation rate experiments in Q4 showed that even small parameter changes have a meaningful effect on convergence, and the scheduling problem in Q5 highlighted how the repair operator can make or break a GA implementation. The real-world comparison in Q6 was a useful reminder that "more sophisticated" does not always mean "better for this problem": for a $6 \times 6$ grid, RRHC with a well-chosen neighbourhood outperformed the GA on both quality and consistency.